

CHAPTER 4: DISCRETIONARY ACCESS CONTROLS

Le Thi Bao Thu

Faculty of Computer Science & Engineering

HCMC University of Technology

thule@hcmut.edu.vn

Outline

-
- 1 Introduction to DAC**
 - 2 Security Models
 - 3 Proposed Models for DAC
 - 4 SQL for Data Control
 - 5 Advantages & Disadvantages
-

Introduction to DAC (1)

- ❑ **DAC: Discretionary Access Control.**
- ❑ User can protect what they own.
- ❑ The owner is given all privileges on their own data.
- ❑ The owner can define the type of access (read/write/execute/...) and grant access to others.
- ❑ The typical method of enforcing **DAC** in a **database system** is based on **granting** and **revoking privileges**.

Introduction to DAC (2)

❑ Types of Discretionary Privileges

- **The account/system level:** privileges are independent of the objects in the database system, granted to users by administrators.
 - CREATE/ALTER/DROP TABLE, CREATE/ALTER/DROP VIEW
 - CREATE /ALTER/DROP USER
 - CREATE SCHEMA, CREATE TRIGGER
 - MODIFY, SELECT
- **The object level:** privileges are used to control access to each individual object in the database system.
 - Objects: relations, views, functions/procedures,...
 - Types of access: SELECT, INSERT, UPDATE, DELETE, REFERENCE,...

Outline

-
- 1 Introduction to DAC
 - 2 Security Models**
 - 3 Proposed Models for DAC
 - 4 SQL for Data Control
 - 5 Advantages & Disadvantages
-

Security Models

- ❑ Provide a semantically rich representation allowing functional and structural properties of the security system to be described.
- ❑ Give a high-level definition of the protection requirements and system policies.
- ❑ Be a software-independent, concept model.
- ❑ Enable to demonstrate that a security system satisfies some properties by proving that the abstract model satisfies those properties.
- ❑ Types of Security Models:
 - Discretionary Models.
 - Non-discretionary Models (or Mandatory Models).

Outline

-
- 1 Introduction to DAC
 - 2 Security Models
 - 3 Proposed Models for DAC**
 - 4 SQL for Data Control
 - 5 Advantages & Disadvantages
-

Discretionary Security Models

- ❑ Govern the access of a user to the information on the basis of:
 - The user's identity.
 - Rules that specify the types of access the user is allowed for each object.
- ❑ Authorization module: check the request of a user to access an object.
- ❑ Models for DAC:
 - Access Matrix model
 - Take-Grant model
 - *Action-Entity model*
 - *Wood et al. model*

Access Matrix Model (1)

- ❑ Be used for both operating system level and database level.
- ❑ Be originally formulated by Lampson (1971) and extended by Graham and Denning (1972).
- ❑ 1976, Harrison and his colleagues presented *Access Matrix Model* systematically.

Access Matrix Model (2)

- ❑ Authorization state: $Q = (S, O, A)$
- S (Subjects): the set of subjects are **active** entities using system resources.
 - E.g.: users, user groups, processes,...
 - O (Objects): the set of objects are entities needing to be protected. They include **passive** entities (system resources) and **active** entities.
 - E.g. (at OS level): files, memories, segments, processes,...
 - E.g. (at DB level): databases, relations, attributes, records, fields,...
 - A (Access matrix): each entry $A[s, o]$ contains the **access modes** for which subject s is authorized on object o .

Access Matrix Model (3)

- For DBs: $A[s,o]$ includes conditions that must be satisfied in order for s to exercise the access modes.

	O_1	...	O_i	...	O_m
S_1	$A[s_1,o_1]$		$A[s_1,o_i]$		$A[s_1,o_m]$
...					
S_i	$A[s_i,o_1]$		$A[s_i,o_i]$		$A[s_i,o_m]$
...					
S_n	$A[s_n,o_1]$		$A[s_n,o_i]$		$A[s_n,o_m]$

■ Possible conditions

- **Data-dependent**: constraints on the value of the accessed data.
E.g.: read only employees whose salary is < 1000 .
- **Time-dependent**: constraints on the time an access can take place.
E.g: can only read between 8:00am and 5:00pm.
- **Context-dependent**: constraints on combinations of accessed data.
E.g.: cannot access 2 fields “name” & “salary” together.
- **History-dependent**: constraints dependent on previously performed accesses.
E.g.: A subject may read “salary” only if he has not previously read “name”.

.....

Access Matrix Model (4)

	Asset 1	Asset 2	File	Device
Role 1	read, write, execute, own	execute	read	write
Role 2	read	read, write, execute, own		

Access Matrix Model (5)

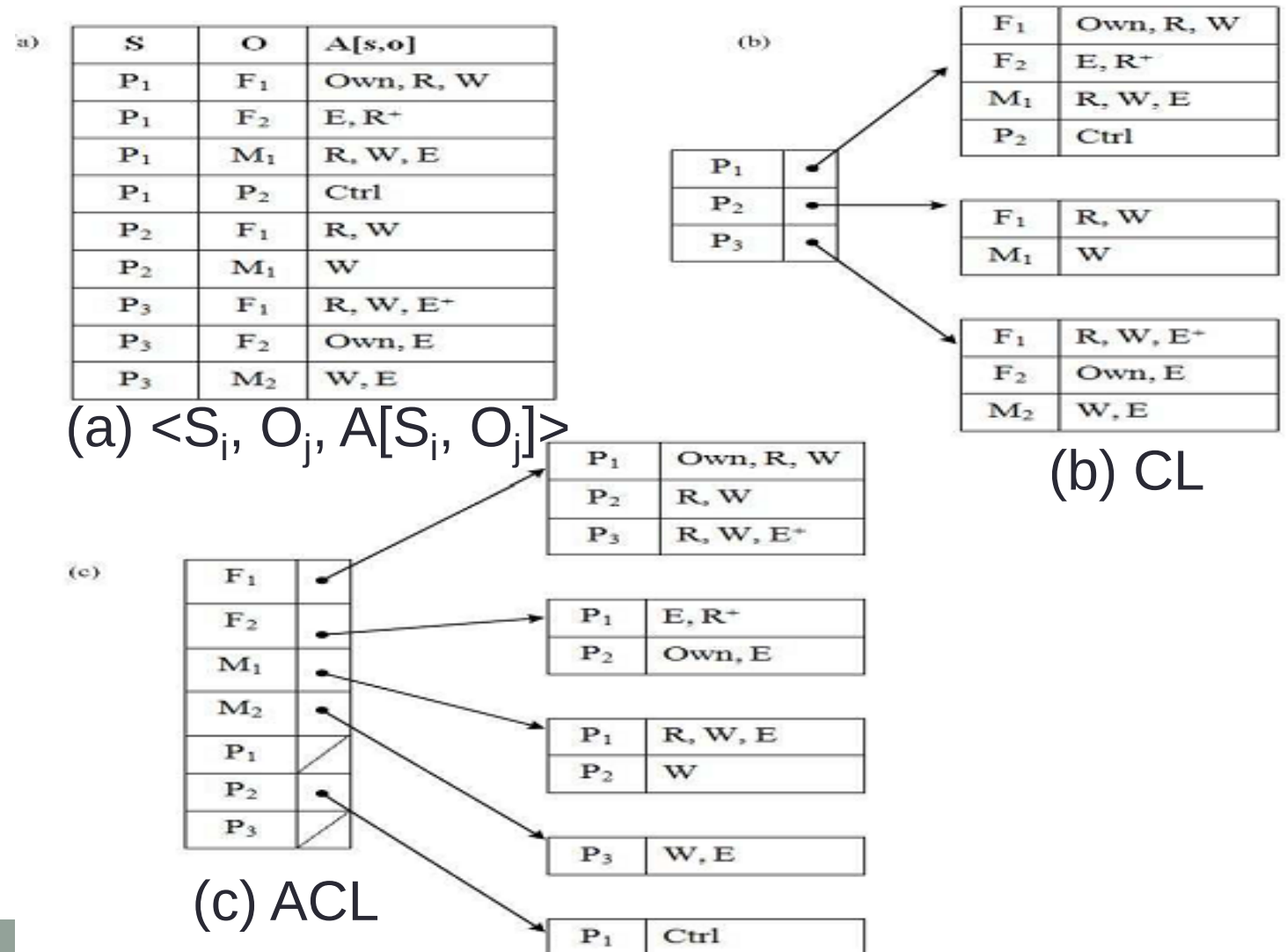
❑ Model implementation

- $S \rightarrow \{(O, A)\}$: Capability List (CL)
 - $Alice \rightarrow \{(file\ X, \{read, delete\}), (file\ Y, \{update\})\}$
- $O \rightarrow \{(S, A)\}$: Access Control List (ACL) $\rightarrow$ Each entry in the list specifies a subject and operation(s):
 - $File\ X \rightarrow \{(Alice, \{read, delete\}), (Bob, \{read\})\}$

❑ Comparison

- CL: compute a set of subjects granted access on a given object $\rightarrow$ all lists must be gone through.
- ACL: find all objects a subject can access.

Access Matrix Model (6)



Take-Grant Model (1)

- ❑ Proposed by *Jones et al.* in 1976.
- ❑ Use a graph structure to represent the authorizations between:
 - Subject and object.
 - Subject and subject.
 - Object and object.
- ❑ Can be represented by its adjacency matrix.
- ❑ Be used to protect OS and database environments.

Take-Grant Model (2)

❑ Authorization state: $G = (S, O, E)$

- $V = S \cup O$: the set of vertexes, $S \cap O = \Phi$
- E : the set of labelled arcs.
- Subjects (active objects): *users, processes, and programs acting on behalf of users.*
- Objects (passive objects): *OS resources (files, memory segments,...) and database resources (databases, relationships, records, fields).*

❑ Symbols

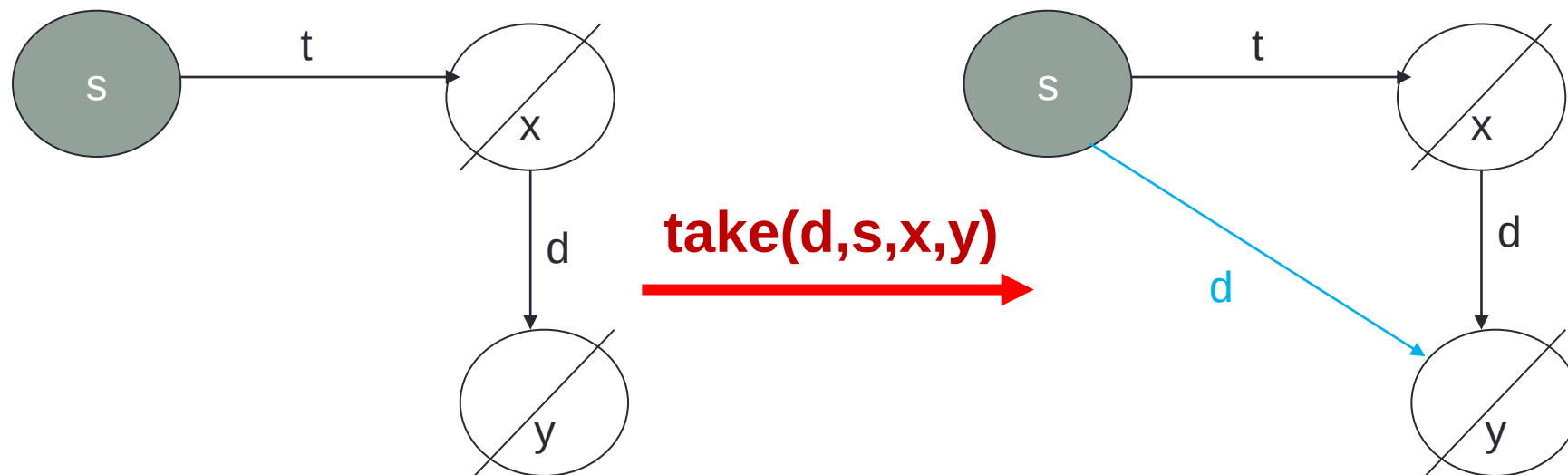
- Filled-in circle: subject.
- White circle: object.
- Barred circle: generic node (either a subject or an object).

Take-Grant Model - Access Modes

- ❑ Read ($x \xrightarrow{r} y$): *Subject* (or *object*) x can **read** the information contained in the object y .
- ❑ Write ($x \xrightarrow{w} y$): *Subject* (or *object*) x can **write** into the object y .
- ❑ Take ($x \xrightarrow{t} y$): *Subject* x is enabled to **take** any rights held by y on other objects or subjects in the system.
- ❑ Grant ($x \xrightarrow{g} y$): *Subject* x can **grant** to y any of the rights held by x on other objects or subjects in the system.

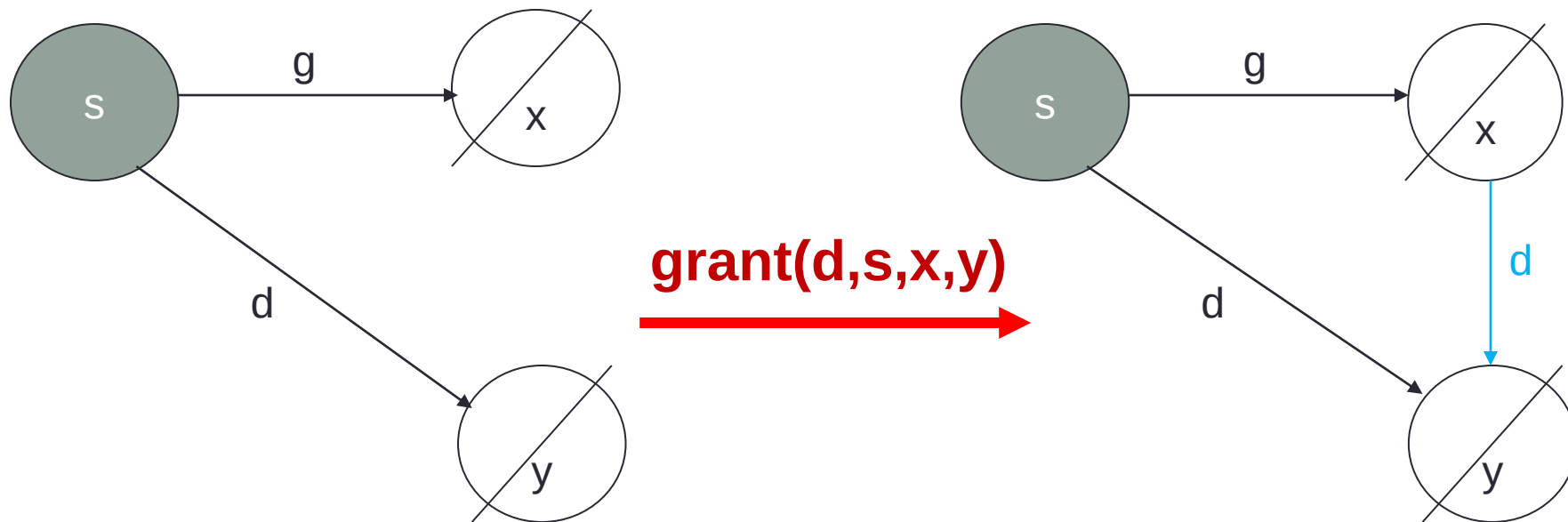
Take-Grant Model - Operations (1)

- ❑ $take(d, s, x, y)$: the subject s **takes** the right d on the object/subject y from the object/subject x .
- ❑ **Conditions ???**



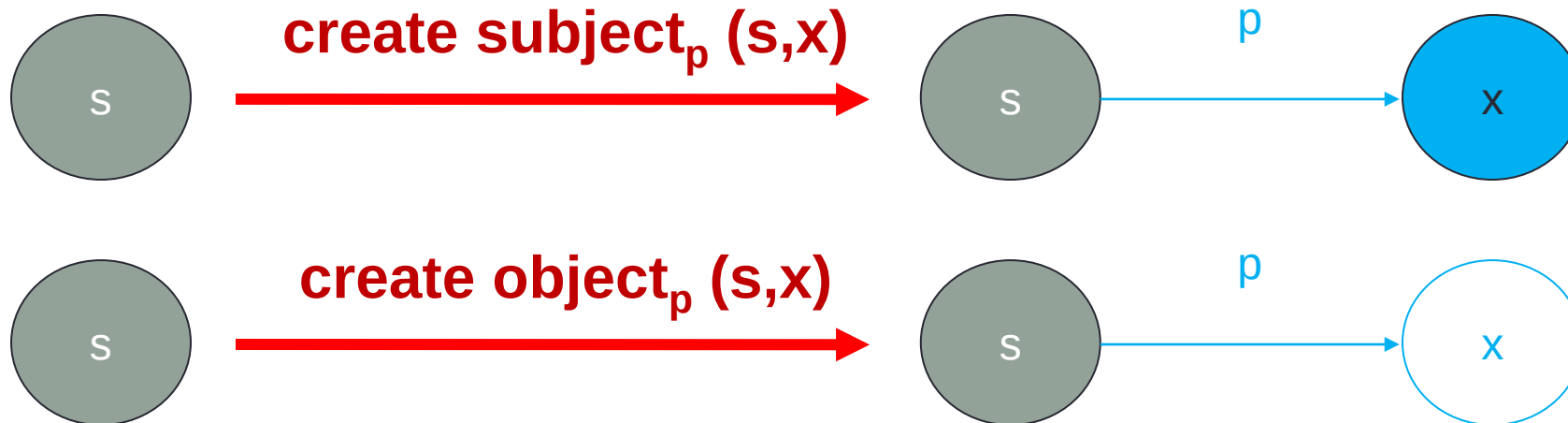
Take-Grant Model - Operations (2)

- ❑ $grant(d, s, x, y)$: the subject s **grants** the right d on the object/subject y to the object/subject x .
- ❑ **Conditions ???**



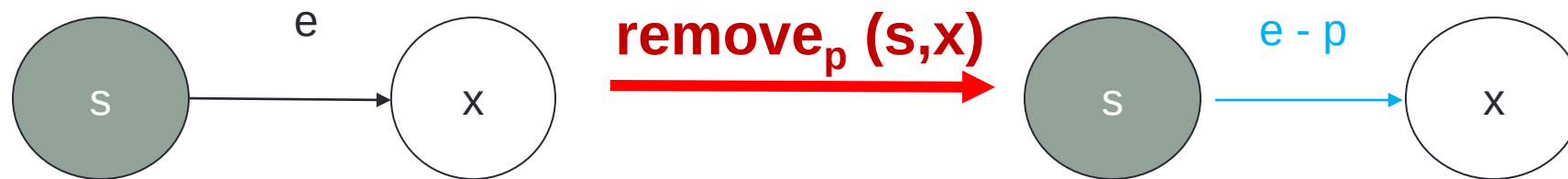
Take-Grant Model - Operations (3)

- *create* $\{subject_p, object_p\}$ (s,x): the subject s **creates** subject/object x and **is granted** the set of rights p on x.



Take-Grant Model - Operations (4)

- ❑ $remove_p(s, x)$: the subject s is **revoked** the p rights it has been granted on subject/object x .



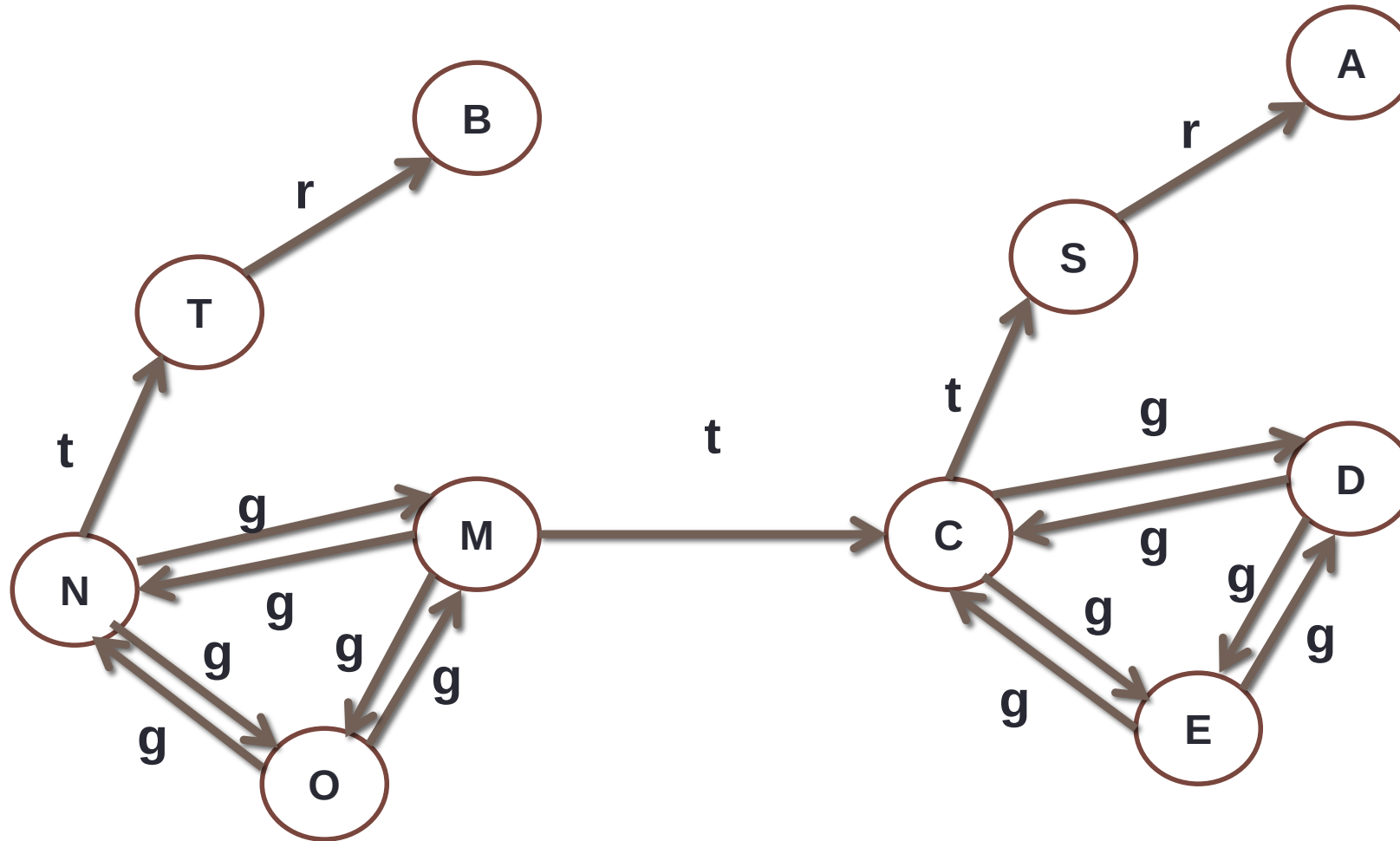
Take-Grant Model – Types of Rights

- ❑ Inert rights (read, write) do not modify the system authorization/protection state.
- ❑ Transport rights (take, grant) modify the system authorization/protection state.
- ❑ Disadvantages?

Take-Grant Model - Disadvantages

- ❑ ***Non-selectivity of administrative rights***
 - All authorizations of S owning a 'GRANT' authorization can be transferred.
 - All authorizations of O/S on which a 'TAKE' right is held can be taken.
- ❑ ***No control on propagation of authorizations:*** A subject S may lose control on the subsequent transfers of the rights it has granted to an object/subject.
- ❑ ***Non-locality:*** S owning the grant privilege on O can give any of its privileges to O, thus augmenting the domain of O (the set of authorizations associated to O).
- ❑ ***Reversibility of the privileges transport flow:*** if the rights can be transferred (directly or indirectly) from x to y, a possible opposite flow from y to x cannot be prevented.

Take-Grant Model - Example (1)



Take-Grant Model - Example (2)

- ☐ Can C, D, E read A?
- ☐ Can M, N, O read A?
- ☐ Can C, D, E read B?
- ☐ Can S read B?
- ☐ Can T read A?

Other Models

- ❑ Acten (Action-Entity) model
- ❑ Wood et al. model
- Read ***Database Security***, S. Castano, M. Fugini, G. Martella, and P. Samarati (1995), ACM Press & Addison-Wesley, ISBN 0-201-59375-0.

Outline

-
- 1 Introduction to DAC
 - 2 Security Models
 - 3 Proposed Models for DAC
 - 4 SQL for Data Control**
 - 5 Advantages & Disadvantages
-

SQL for Data Control (1)

- ❑ Commands: GRANT, REVOKE

- ❑ Based on:

- Users.
- Database objects.
- Privileges on a relation R:
 - SELECT: read data from R.
 - Modify (INSERT, UPDATE, DELETE): modify data of R. INSERT and UPDATE can be granted on some attributes of R (instead of an entire row),
 - REFERENCE: reference to R (foreign key). REFERENCE can also be granted on some attributes of R.

SQL for Data Control (2)

- ❑ GRANT: **pass** privileges on database objects to other users.

GRANT <privilege list>
ON <database objects>
TO <user list>

- ❑ REVOKE: **take back (cancel)** privileges on database objects from other users.

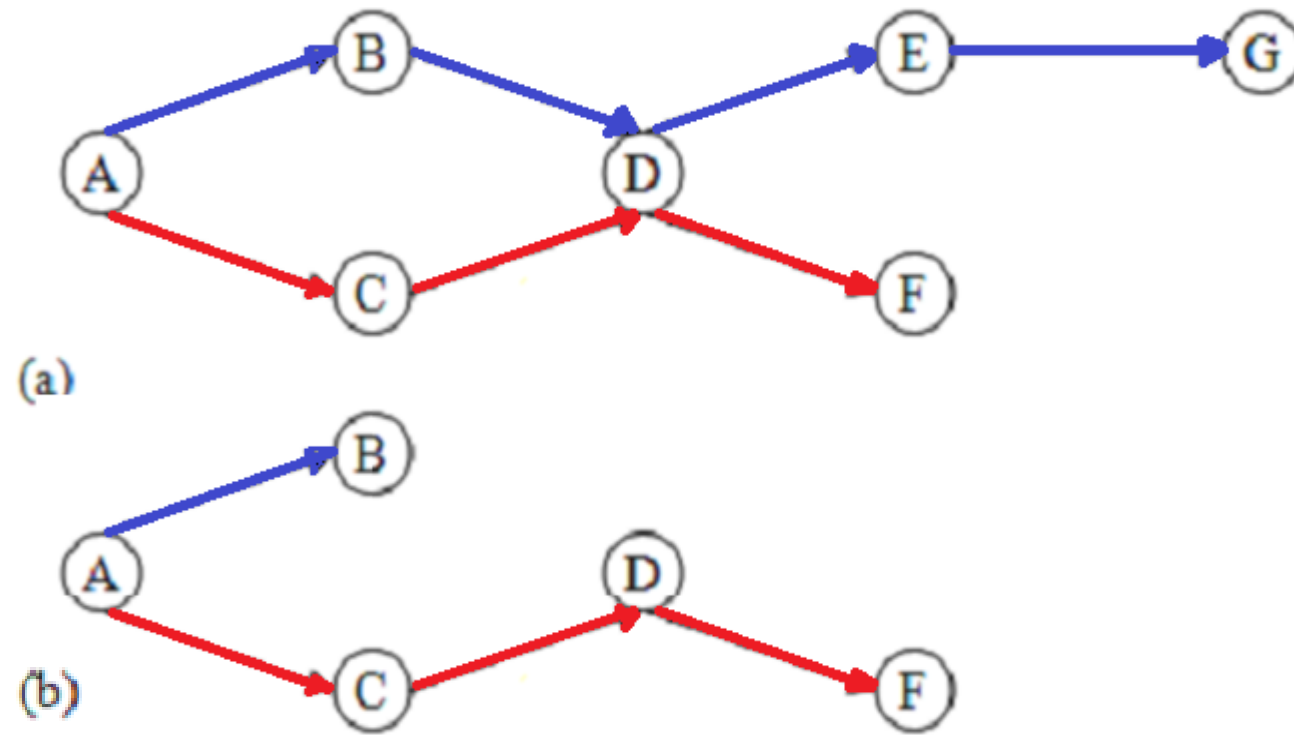
REVOKE <privilege list>
ON <database objects>
FROM <user list>

SQL for Data Control (3)

❑ Propagation of privileges using the GRANT OPTION

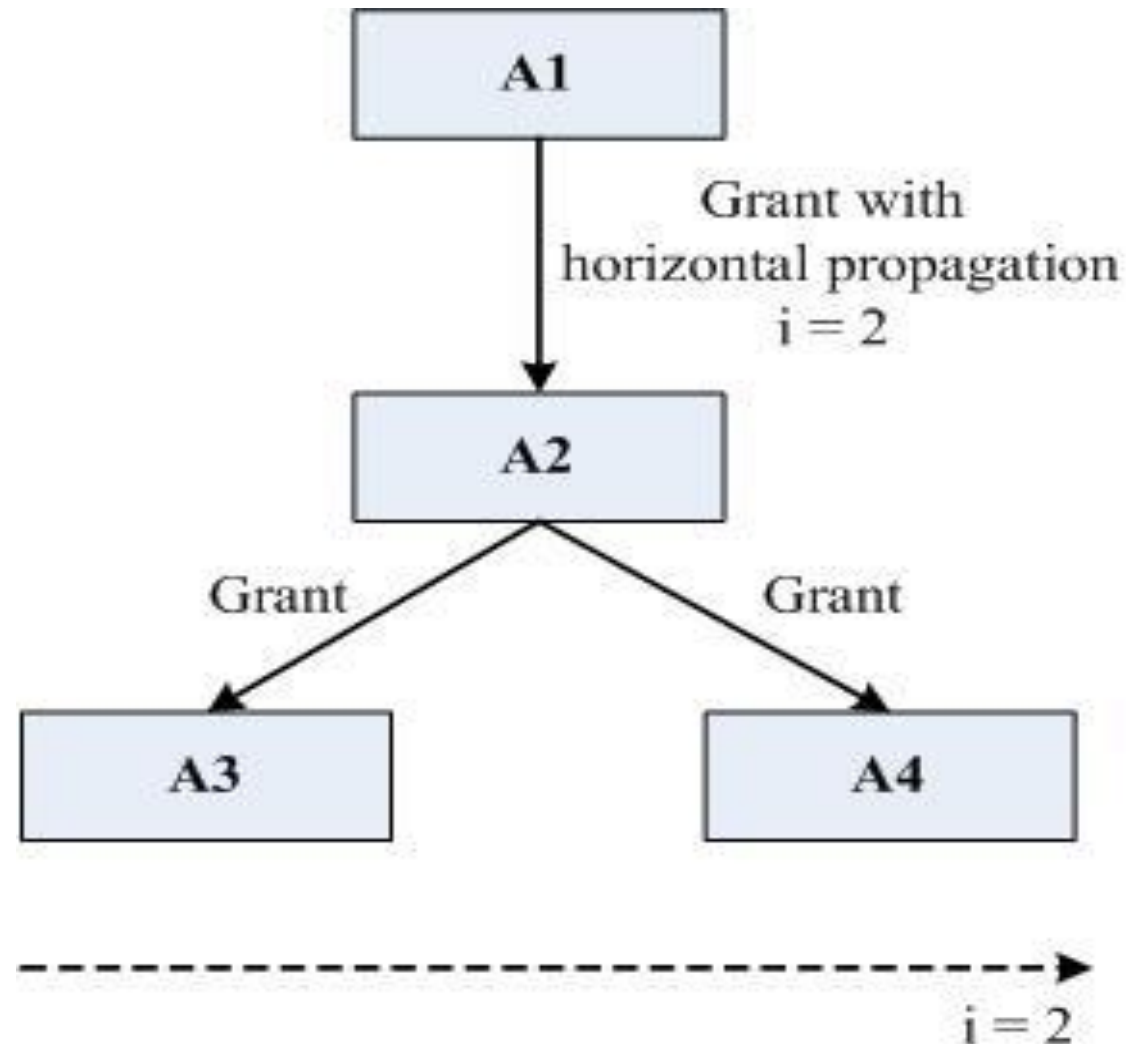
- Whenever the owner A of a relation R grants a privilege on R to another account B, this privilege can be given to B with or without the GRANT OPTION.
 - If the GRANT OPTION is given, this means that B can also grant that privilege on R to other accounts with or without GRANT OPTION.
- ➔ The owner A does not know about the propagation of that privilege on R to other users.

SQL for Data Control (4)



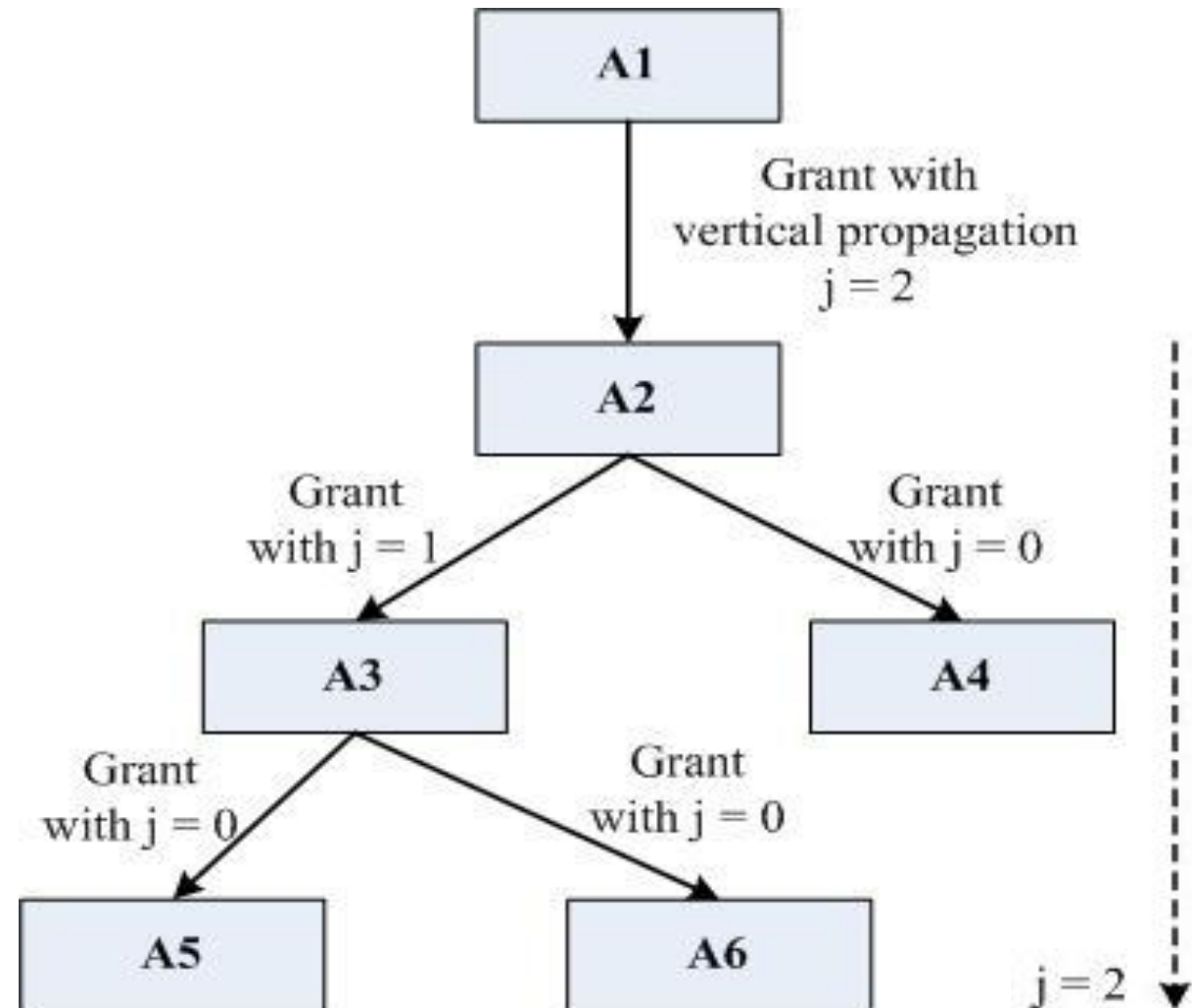
SQL for Data Control (5)

Limit
horizontal
propagation



SQL for Data Control (6)

Limit vertical propagation



SQL for Data Control (7)

❑ DAC with **views** (virtual relations)

- If the owner A of a relation R wants another account B to be able to retrieve only some fields of R, then A can create a view V of R that includes only those attributes and then grant SELECT on V to B.
- The same applies to limit B to retrieve only certain tuples of R: a view V' can be created by defining a query that selects only those tuples from R and then grant SELECT on V' to B.

SQL for Data Control - An Example (1)

- ❑ Suppose that the DBA creates 4 accounts: A1, A2, A3, A4.
- ❑ For only A1 to be able to create base relations, DBA must issue the following GRANT command in SQL:

GRANT CREATETAB TO A1;

- ❑ DBA creates schema EXAMPLE and grants privileges on it to A1:

CREATE SCHEMA EXAMPLE AUTHORIZATION A1;

SQL for Data Control - An Example (2)

- ❑ A1 can create tables under the schema EXAMPLE.
- ❑ A1 creates the 2 relations EMPLOYEE and DEPARTMENT so that A1 is the owner of these relations and hence have all the relation privileges on each of them.

EMPLOYEE

Name	<u>Ssn</u>	Bdate	Address	Sex	Salary	Dno
------	------------	-------	---------	-----	--------	-----

DEPARTMENT

<u>Dnumber</u>	Dname	Mgr_ssn
----------------	-------	---------

SQL for Data Control - An Example (3)

- ❑ A1 wants to grant A2 the privileges to **insert and delete** tuples in both of these relations but does **not want A2 to be able to propagate these privileges** to other accounts:

GRANT INSERT, DELETE

ON EMPLOYEE, DEPARTMENT TO A2;

- ❑ A1 wants to allow A3 to retrieve information from either of the two tables and also to be able to propagate the SELECT privilege to other accounts.

GRANT SELECT ON EMPLOYEE, DEPARTMENT

TO A3 WITH GRANT OPTION;

SQL for Data Control - An Example (4)

- ❑ A3 can grant the SELECT privilege on the EMPLOYEE relation to A4:

GRANT SELECT ON EMPLOYEE TO A4;

- ❑ Notice that A4 cannot propagate the SELECT privilege because GRANT OPTION was not given to A4.

SQL for Data Control - An Example (5)

- ❑ A1 decides to revoke the SELECT privilege on EMPLOYEE from A3:

REVOKE SELECT ON EMPLOYEE FROM A3;

- ❑ The DBMS must now automatically revoke the SELECT privilege on EMPLOYEE from A4, because A3 granted that privilege to A4 and A3 does not have the privilege any more.

SQL for Data Control - An Example (6)

- ❑ A1 wants to give back A3 a limited capability to SELECT from the EMPLOYEE relation and allow A3 to be able to propagate the privilege.
 - The limitation is to retrieve only the NAME, BDATE, and ADDRESS attributes and only for the tuples with DNO=5.

- ❑ A1 then creates the view:

```
CREATE VIEW A3_EMPLOYEE AS  
SELECT NAME, BDATE, ADDRESS  
FROM EMPLOYEE WHERE DNO = 5;
```

- ❑ After the view is created, A1 can grant SELECT on the view A3_EMPLOYEE to A3 as follows:

```
GRANT SELECT ON A3_EMPLOYEE  
TO A3 WITH GRANT OPTION;
```

SQL for Data Control - An Example (7)

- ❑ A1 wants to allow A4 to update only the SALARY attribute of EMPLOYEE:
GRANT UPDATE ON EMPLOYEE (SALARY) TO A4;
- ❑ The UPDATE and INSERT privileges can specify particular attributes that may be updated or inserted in a relation.
- ❑ Other privileges (SELECT, DELETE) are not attribute specific.

Outline

-
- 1 Introduction to DAC
 - 2 Security Models
 - 3 Proposed Models for DAC
 - 4 SQL for Data Control
 - 5 Advantages & Disadvantages**
-

Advantages & Disadvantages

❑ Advantages

- Govern the access to the information on the basis of discretionary protection policies → flexible.
- Suitable for various types of systems and applications.
- Be widely used in a variety of implementations, especially in the commercial and industrial environments.

❑ Disadvantages

- Allow information from an object which can be read by a subject to be written to any other objects.
 - E.g.: Bob is denied access to file A, so he asks cohort Alice to copy A to B that he can access.
- Suppose the users are trusted not to do this deliberately, it is still possible for Trojan Horses to copy information from one object to another.

Trojan Horse Example (1)

User Alice

r: Alice; w:Alice

File A

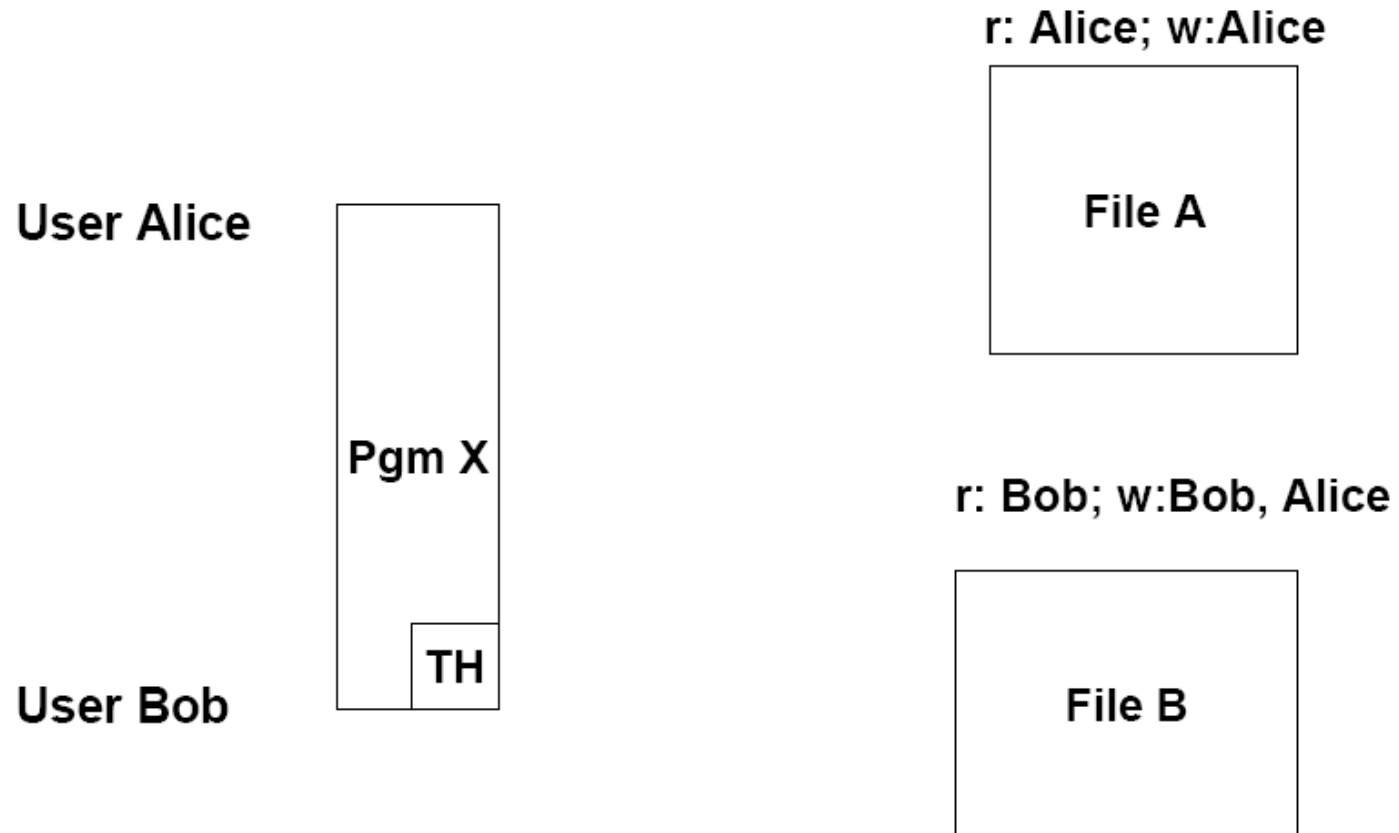
User Bob

r: Bob; w:Bob

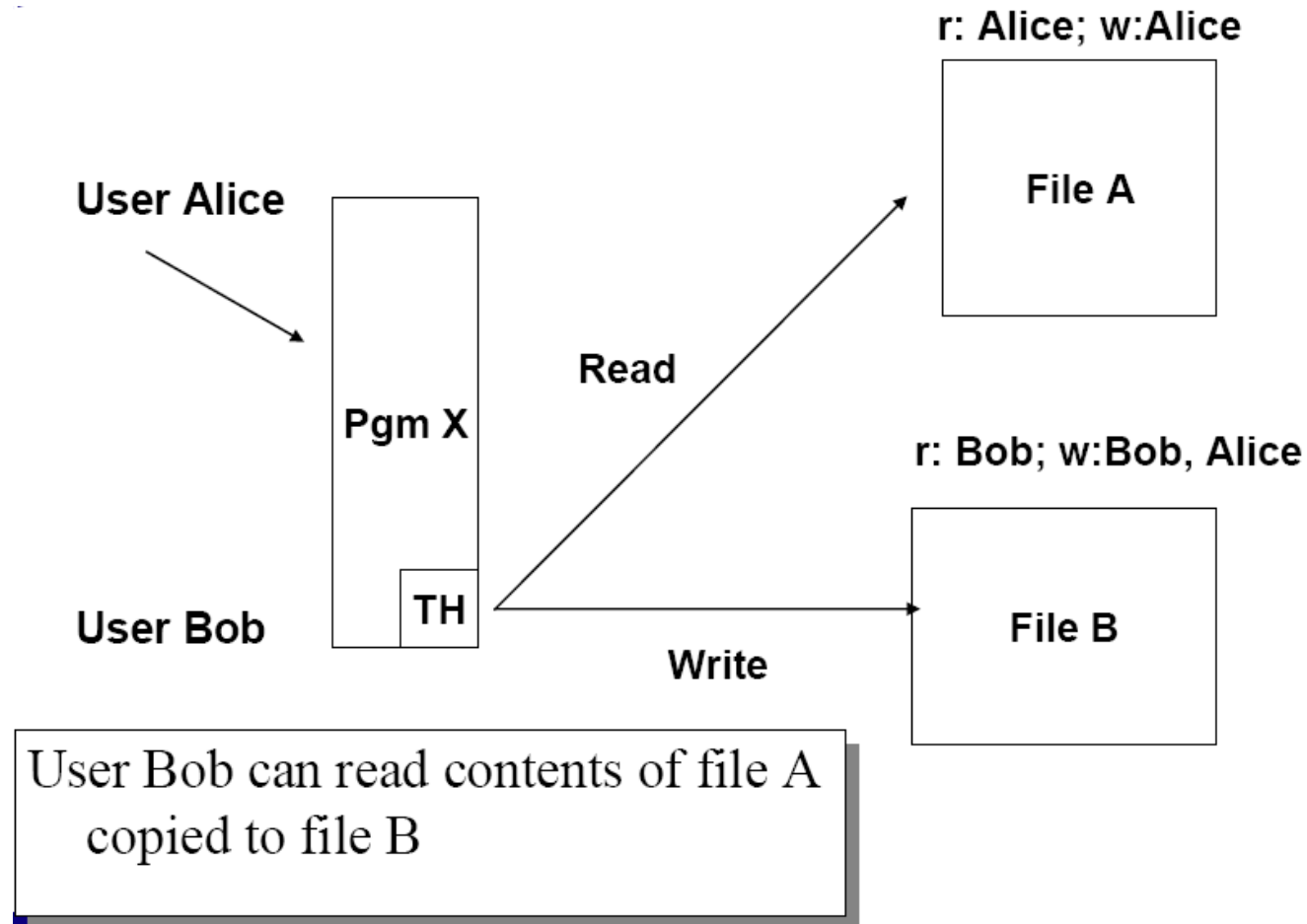
File B

User Bob cannot read the file A!

Trojan Horse Example (2)



Trojan Horse Example (3)



Outline

-
- 1 Introduction to DAC
 - 2 Security Models
 - 3 Proposed Models for DAC
 - 4 SQL for Data Control
 - 5 Advantages & Disadvantages
-

